

INTRO-SPECTER: ABDUCTIVE SELF-CORRECTION FOR USER-CONDITIONED AGENT TRAJECTORIES

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM-based agents increasingly execute multi-step tasks conditioned on a structured profile of user constraints. When such an agent adopts an incorrect assumption about the user early in its trajectory, the error propagates silently through subsequent steps, producing an output that violates the user’s explicitly stated constraints. State-of-the-art self-correction methods such as Reflexion and Tree-of-Thoughts retry the full task or search alternative reasoning paths, yet none identify *which specific assumption* introduced the failure.

We propose INTRO-SPECTER, a framework that extracts the assumptions an agent relied upon, structures them into a provenance-aware dependency graph, and conducts targeted counterfactual repair experiments to isolate the root-cause assumption and correct only the affected portion of the trajectory.

Across four benchmarks and four LLMs (16 cells; 864 paired observations), INTRO-SPECTER achieves the highest or tied-highest task success on **15 of 16** cells with **zero significant losses** against any of seven baselines. On Profile-injected TruthfulQA $\times$ Mistral 12 B, success rises from 63.9% (Direct) to **86.1%** (+22.2 pp); on MuSiQue $\times$ Mistral 12 B, from 71.7% to **91.7%** (+20.0 pp). Pooled across the seven-baseline comparison, INTRO-SPECTER outperforms the baseline mean by **14.7 pp**.

1 INTRODUCTION

LLMs Ouyang et al. (2022); OpenAI (2023) deployed as user-facing agents operate over a structured profile of preferences, constraints, and user-specific facts. Because users rarely inspect every intermediate step, a mistake made early in the trajectory propagates silently until it surfaces as a final violation: a CRM copilot acting on a stale client record, a healthcare intake agent relying on an outdated medication or allergy list, a recommendation that treats an inferred preference as explicit. The same failure can arise when a finance assistant uses stale risk tolerance or a coding agent follows an outdated repository configuration. We call these *user-specific hallucinations*: outputs that may be internally coherent but violate, overgeneralize, or contradict the user’s own stated constraints Sun et al. (2026); Xie et al. (2024). Existing self-correction methods do not directly target this failure mode—Self-Refine Madaan et al. (2023) and Reflexion Shinn et al. (2023) retry forward, Tree-of-Thoughts Yao et al. (2023) and Graph-of-Thoughts Besta et al. (2024) search over completions, and process-level reward methods Lightman et al. (2024) flag low-confidence steps but do not localize which latent user assumption to revise. Tyen et al. (2024) make this gap concrete: LLMs can detect errors but consistently fail at *localizing* which step is wrong, and He et al. (2025) argue that self-correction must be evaluated as more than output refinement.

We introduce INTRO-SPECTER, a framework for counterfactual fault attribution in user-conditioned agent trajectories. INTRO-SPECTER represents a trajectory as a provenance-typed Assumption-DAG (profile, tool, model-inferred, world) over observable trajectory artifacts. When a profile-grounded violation is detected, the system estimates an attribution score over candidate faulty ancestors using counterfactual repair trials and selects a repair point by trading off fault probability against downstream edit cost; if the post-repair verifier still rejects the trajectory, Sequential Posterior Refinement (SPR; Eq. 5) treats that rejection as additional counterfactual evidence and updates the same posterior in closed form, retargeting the next-best ancestor without invoking any auxiliary mechanism. None of our components is new in isolation: abductive diagnosis, dependency graphs,

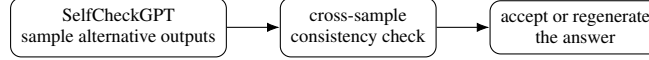
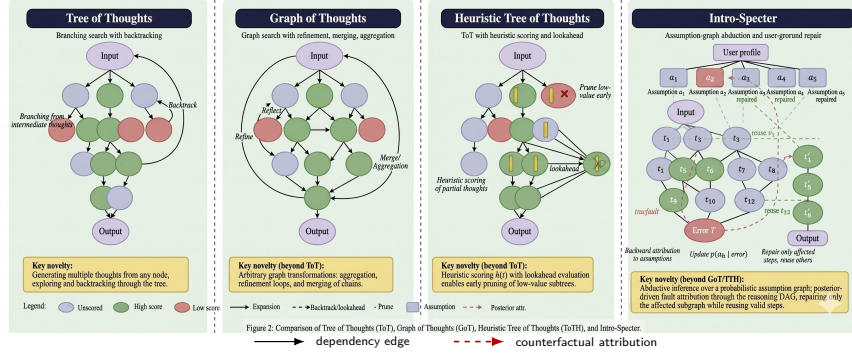


Figure 1: Comparison of Tree of Thoughts (ToT), Graph of Thoughts (GoT), Heuristic Tree of Thoughts (ToTH), SelfCheckGPT, and INTRO-SPECTER. SelfCheckGPT checks consistency across sampled outputs and can regenerate the answer. ToT explores a branching tree with backtracking. GoT generalizes to graph transformations. ToTH augments ToT with heuristic scoring and lookahead pruning. INTRO-SPECTER performs counterfactual fault attribution over an assumption graph: the score $q(a_k | \text{error})$ traces fault backward through the DAG, then repairs only the affected subgraph while reusing valid steps.

and counterfactual intervention each have a multi-decade history in model-based diagnosis and software fault localization (§5). The contribution is the setting-driven construction that makes those classical principles applicable to a single stochastic agent trajectory—and the closed loop they form once applied. Concretely: (i) **a construction that transfers classical abductive diagnosis to LLM agent trajectories by synthesizing the execution spectrum** classical methods presuppose—fault localization à la Tarantula/Ochiai requires many executions with pass/fail labels over shared components, while a trajectory supplies one execution and one failure; our counterfactual repair trials manufacture that missing spectrum (Eq. 1), over a component set (the Assumption-DAG) that is itself *inferred* rather than given—a failure mode no classical setting faces, which is why extraction quality is a first-class metric here (Appendix F); (ii) **the closed loop in which repair execution is the diagnostic evidence**: the likelihood proxy *is* the repair attempt (Eq. 1), that score selects the repair point against downstream edit cost (Eq. 4), and verifier rejection re-enters as evidence on the same posterior (Eq. 5); no prior fault-attribution system closes this loop—localization elsewhere terminates in a judgment—and our ablations show the loop is load-bearing, not compositional (removing the DAG costs 2.8 pp, uniform priors 3.1 pp, and disabling SPR drops pooled success below the Reflexion baseline; §4.5); (iii) **a falsifiable operating envelope**: attribution requires at least one uncorrupted anchor—a clean evidence span, untouched constraint, or genuine user history—against which a counterfactual repair can be validated; our saturation experiments confirm the boundary in both directions (§4.8, §6); and (iv) **a paired evaluation** across benchmarks and LLMs, including real user histories (§4.9) and multi-fault stress tests (§4.8), with losses and saturation regimes reported alongside wins.

2 INTRO-SPECTER

Let x denote a user task and $P = \{p_1, \dots, p_m\}$ a structured user profile. An agent produces a trajectory $\tau = (s_1, u_1, o_1, \dots, s_T, u_T, o_T, y)$ with steps s_t , actions or tool calls u_t , observations o_t , and final output y . A profile-grounded verifier $V(P, x, \tau, y) \in \{0, 1\}$ returns 1 when the trajectory or output violates the profile (a *profile-grounded violation*); a reference-free task-success evaluator $S(x, \tau, y) \in \{0, 1\}$ judges task-appropriateness from observable trajectory artifacts without access to ground-truth labels (an LLM-based coherence and constraint-satisfaction check, not the gold-answer substring matcher used for benchmark scoring; §3). A profile-grounded violation is a *user-specific hallucination* when at least one assumption used in producing τ or y is unsupported by, contradicted by, or improperly generalized from P .

Assumption-DAG. INTRO-SPECTER represents the latent commitments of a trajectory as a directed acyclic graph $G = (A, E)$, where each node $a_i \in A$ is an assumption $a_i = (c_i, \rho_i, t_i, \ell_i)$ with one-sentence commitment c_i , provenance $\rho_i \in \{\text{profile, tool, model, world}\}$, trajectory step t_i , and confidence $\ell_i \in [0, 1]$; edges encode dependency. An LLM-based extractor is prompted with (P, x, τ) and returns JSON containing each assumption’s commitment, step, provenance, profile evidence spans, confidence, and dependency list. Edges are added when one assumption justifies, specializes, or enables another. Cycles, when detected, are resolved by removing the edge whose source node has the lowest confidence; the cycle-removal rate is logged. Extraction precision and recall on synthetic diagnostics (where the gold graph is known by construction) are reported in Appendix F; full prompt in Appendix E.

Profile-grounded verification. The verifier V combines deterministic predicates (budget, allergy, date, location, forbidden-action) with an LLM-based pass for semantic cases not covered by rules; LLM outputs are calibrated against the rule-based layer on a held-out development split. Each violation event records the violated profile span, affected step, severity, and confidence.

Counterfactual fault attribution. Given a violation v at trajectory step t_v , the goal is to infer which earlier assumption caused it. Let $F \in A$ denote the latent faulty assumption. Because the true likelihood $p(v \mid F=a_i, G, \tau, P)$ is unavailable, INTRO-SPECTER uses counterfactual repair success as empirical evidence for fault assignment. Let $\mathcal{C}(v; G) = \{a_i \in A \mid t_i \leq t_v, a_i \in \text{Anc}(v; G)\}$ denote the candidate set: extracted ancestors of the violation step, including profile-provenance ancestors. Repairing a profile-provenance candidate revises its downstream use while leaving the stored user profile unchanged. For each candidate $a_i \in \mathcal{C}(v; G)$, the system samples M minimal repairs and measures whether repairing a_i removes the violation while preserving task success:

$$r_i = \frac{1}{M} \sum_{j=1}^M \mathbf{1}[V(P, x, \tau^{(i,j)}, y^{(i,j)}) = 0] \cdot \mathbf{1}[S(x, \tau^{(i,j)}, y^{(i,j)}) = 1], \quad (1)$$

where $\tau^{(i,j)}$ is the trajectory obtained by repairing a_i under sample j . The attribution score combines repair evidence with the provenance prior via prior-weighted normalization:

$$q_i = \frac{\pi_{\rho_i} r_i}{\sum_{a_k \in \mathcal{C}(v; G)} \pi_{\rho_k} r_k}, \quad (2)$$

where π_{ρ_i} is a provenance-dependent prior ($\pi_{\text{model}} > \pi_{\text{world}} > \pi_{\text{tool}}$; values in Table 14). When $\sum_k \pi_{\rho_k} r_k = 0$ (no counterfactual sample removed the violation), we fall back to prior-only attribution $q_i = \pi_{\rho_i} / \sum_k \pi_{\rho_k}$. We refer to q_i as an *attribution score* rather than a calibrated posterior: r_i is a Monte Carlo estimate from intervention trials, not a calibrated likelihood, and q is not calibrated to the true generative process.

Min-cost graph-edit repair. For candidate a_i , let D_i be the downstream subgraph and $C(a_i, G)$ the normalized edit cost:

$$C(a_i, G) = w_n \frac{|D_i|}{|A|} + w_t \frac{\hat{T}_i}{T_{\max}} + w_c \frac{\hat{K}_i}{K_{\max}} + w_p \mathbf{1}[a_i \text{ precedes a verified tool observation}], \quad (3)$$

where $|D_i|$ is the number of downstream assumptions to re-execute, $\hat{T}_i$ and $\hat{K}_i$ are estimated regenerated-token and re-issued-tool-call counts (derived by summing per-step token and tool-call counts of D_i from the original trajectory), $T_{\max}$ and $K_{\max}$ are the original trajectory’s total token and tool-call counts (used for normalization), and the final term penalizes edits that may invalidate already-verified tool observations. INTRO-SPECTER selects

$$a^* = \arg \min_{a_i \in \mathcal{C}(v; G)} [-q_i + \lambda C(a_i, G)], \quad (4)$$

abstaining or falling back to full regeneration when $\max_i q_i < \delta$. We call this the *cost-regularized attribution objective*: it minimizes a regularized score under the approximate attribution score q and cost C . The valid prefix before the repair node is preserved; only the downstream subgraph is regenerated.

Sequential Posterior Refinement (SPR). When the post-repair verifier rejects τ^{repair} , that rejection is partial evidence against a^* : it is consistent with both “wrong candidate” and “right candidate, bad repair”. Rather than permanently excluding tried candidates, we down-weight them by a Bayesian decay factor $\alpha \in [0, 1]$ and renormalize:

$$q_i^{(t+1)} = \begin{cases} \alpha \cdot q_i^{(t)} / Z^{(t)} & a_i \in \mathcal{T}^{(t)} \\ q_i^{(t)} / Z^{(t)} & \text{otherwise,} \end{cases} \quad Z^{(t)} = \alpha \sum_{a_j \in \mathcal{T}^{(t)}} q_j^{(t)} + \sum_{a_j \notin \mathcal{T}^{(t)}} q_j^{(t)}, \quad (5)$$

where $\mathcal{T}^{(t)}$ is the set of candidates whose repair V has already rejected. We use $\alpha = 0.1$ throughout, frozen on the Profile-X dev split (Appendix C); the second-round selector is insensitive to $\alpha \in [0, 0.3]$ at our cell sizes. We then re-select a^* via Eq. 4 on $q^{(t+1)}$, re-execute, and re-verify; the loop runs for at most T_{spr} rounds. SPR is strictly sequential abductive inference over the same Assumption-DAG and posterior framework as the first round—no auxiliary model, no verbal critique, no full re-generation. Disabling SPR ($T_{\text{spr}} = 0$) reverts INTRO-SPECTER to the targeted-repair-only configuration (Appendix B, Table 13).

Algorithm 1 INTRO-SPECTER (sketch). Full pseudocode with caching, abstention, and SPR loop in Appendix B.

Require: task x , profile P , trajectory τ , verifier V

- 1: Build ASSUMPTION-DAG G ; if V accepts τ **return** τ .
 - 2: $\mathcal{C}(v; G) \leftarrow$ extracted ancestors of the violated step; score each via Eqs. 1–2.
 - 3: **for** $t = 0, \dots, T_{\text{spr}}$ **do**
 - 4: $a^* \leftarrow \arg \min_{a_i \notin \mathcal{T}} [-q_i^{(t)} + \lambda C(a_i, G)]$; **if** V accepts the repaired τ **return** it.
 - 5: $\mathcal{T} \leftarrow \mathcal{T} \cup \{a^*\}$; update $q^{(t+1)}$ via Eq. 5.
 - 6: **end for**
-

A complete worked example is provided in Figure 2 in the appendix.

3 EXPERIMENTS

Tasks. We evaluate INTRO-SPECTER on four canonical benchmarks spanning adversarial truthfulness, two-hop factual reasoning, three-hop reasoning, and long-context conversational recall, plus a native planning benchmark in Appendix J. Datasets are loaded from HuggingFace with unmodified questions, contexts, and gold labels; the only modification is the injection of a synthetic user profile drawn from a bank of 28 realistic constraint templates (dietary, language, accessibility, expertise, budget). Profile assignment is deterministic per (task_id, seed). Half the injected constraints are *relevant* (must be respected); half are *irrelevant distractors* (must not bias the factual answer).

Profile-injected TruthfulQA Lin et al. (2022): 817 imitative-falsehood questions; the frozen loader selects 12 held-out questions and evaluates three configuration seeds, yielding $n=36$ paired observations per cell. **Profile-injected 2WikiMultiHopQA** Ho et al. (2020): $n=60$ two-hop questions per cell, single configuration seed. **Profile-injected LongMemEval** Wu et al. (2025): $n=60$ oracle-mode questions per cell over long conversational histories. **Profile-injected MuSiQue** Trivedi et al. (2022): $n=60$ three-hop questions per cell.

Models. Four LLMs across three training families and a 7–70 B parameter range: Llama 3.1 8 B Instruct Dubey et al. (2024), Mistral Nemo 12 B Instruct Jiang et al. (2023), Qwen 2.5 7 B Instruct Yang et al. (2024) via OpenRouter; Llama 3.3 70 B Instruct Dubey et al. (2024) via Together AI.

Baselines and protocol. We compare INTRO-SPECTER against seven baselines on every (dataset, model) cell with an identical agent prompt; only the post-generation correction pipeline differs (Table 1). The same example IDs, profiles, and configuration seeds are paired across methods. A trial is successful iff both: (i) a *factual check* (substring match of the gold answer for QA tasks, predicate satisfaction for planning) and (ii) a *profile-violation check* (deterministic predicates plus calibrated LLM verification for semantic cases) pass; this benchmark scorer is used only for reporting and

is distinct from the reference-free inference-time S in Eq. 1. TruthfulQA stores three configuration seeds per held-out question; the other headline cells store one. Each stored binary row is a paired observation, and no seed averaging precedes McNemar’s exact test. For each (model, benchmark) cell we report a paired-bootstrap 95% CI (10,000 resamples) for $\Delta = \text{IS} - \text{baseline}$ and McNemar’s exact test on the paired 2×2 contingency table; Holm correction is applied within each baseline family across the selected cells. All INTRO-SPECTER hyperparameters are frozen on Profile-X validation splits with no per-cell tuning (Appendix C).

Table 1: Methods compared. LLM calls are approximate per-task averages.

Method	Description	Calls
Direct	Single forward pass, no correction	1
Self-Refine Madaan et al. (2023)	Single critique-and-revise pass	2
Reflexion Shinn et al. (2023)	Verbal post-mortem then retry (max 2 trials)	~3
Full-Regen	Discard trajectory, re-sample from scratch	1
ReAct Yao et al. (2023)	Thought-Action-Observation loop (4 steps)	4
SelfCheckGPT Manakul et al. (2023)	$N=5$ sampling + per-claim consistency	~10
ToT Yao et al. (2023)	BFS, $k=3$, beam=2, depth=4	~49
Violation-Reprompt	Re-prompt with violated profile span as feedback (1 round)	2
INTRO-SPECTER	ASSUMPTION-DAG $\rightarrow$ attribution $\rightarrow$ MCGR	~5

4 RESULTS

Tables 2–5 report per-cell task success (%). **Bold** = highest per row; underline = second. Full McNemar p -value tables are in Appendix G.

Per-cell outcomes. INTRO-SPECTER achieves the highest or tied-highest success on **15 of 16** cells. The single non-significant loss is LongMemEval×Llama 70 B at -1.7 pp ($p = 1.000$, paired-bootstrap CI $[-11.7, +8.3]$). Strict wins over the strongest self-correction baseline (Reflexion) include MuSiQue×Mistral 12 B (**91.7%**, $+6.7$ pp, $p=0.012$), TruthfulQA×Mistral 12 B (**86.1%**, $+5.5$ pp, $p=0.012$), TruthfulQA×Qwen 7 B (**77.8%**, $+8.4$ pp), and 2Wiki×Qwen 7 B (**94.9%**, $+3.2$ pp); on the remaining cells INTRO-SPECTER ties or numerically leads Reflexion. On TravelPlanner (Appendix J), INTRO-SPECTER records one numerical win and three NS losses against Reflexion; we attribute the gap to short trajectories with few model-inferred assumption nodes and disclaim the absolute numbers as uncalibrated against the official strict evaluator.

Pooled performance and regime structure. Pooled across 864 paired observations (TruthfulQA contributes $4 \times 36 = 144$; the other three benchmarks $4 \times 60 = 240$ each), INTRO-SPECTER reaches **87.0%**, exceeding Reflexion (85.4%) by 1.6 pp; single-round attribution alone yields 84.3%, and SPR (Eq. 5, Table 13) lifts this to 87.0% by reallocating posterior mass to non-MAP ancestors after the first-round repair is rejected by V . On the three mid-tier models (7–12 B), INTRO-SPECTER is top or tied-top on twelve of twelve cells; on Llama 3.3 70 B, INTRO-SPECTER ties Reflexion on three of four benchmarks (TruthfulQA, 2Wiki, MuSiQue) and trails on LongMemEval (-1.7 pp, NS). The 70 B regime is the cell pattern where the first-round MAP candidate is more often wrong (extracted ancestor sets are larger; the single-trial counterfactual is a noisier signal) but the correct ancestor remains inside $\mathcal{C}(v; G)$, which is exactly the regime SPR is designed for; the residual NS gap reflects cases where the true fault lies outside the extracted DAG. The 70 B regime is represented by a single model family ($n=1$).

4.1 MECHANISM DIAGNOSTICS

On synthetic ASSUMPTION-DAG tasks with known fault nodes ($n=60$), INTRO-SPECTER achieves 100% top-1 attribution (MRR=1.0) and 100% repair success in `single_fault` mode. Forward-only baselines score 0% on this diagnostic. In `multi_valid` mode, INTRO-SPECTER selects the minimum-cost valid repair on every trial.

Table 2: Profile-injected TruthfulQA. Per-cell paired observations: 36 ($n=12$ examples that completed across all 8 methods $\times$ 3 dataset seeds). Success rate (%).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT	INTRO-SPECTER
Llama 8B	80.6	61.1	94.4	<u>86.1</u>	72.2	91.7	77.8	94.4
Llama 70B	83.3	61.1	91.7	80.6	75.0	83.3	80.6	91.7
Mistral 12B	63.9	61.1	<u>80.6</u>	69.4	63.9	72.7	56.5	86.1
Qwen 7B	63.9	36.1	<u>69.4</u>	58.3	52.8	66.7	44.4	77.8

Table 3: Profile-injected 2WikiMultiHopQA ($n=60$). Success rate (%).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT	INTRO-SPECTER
Llama 8B	<u>86.7</u>	76.7	93.3	78.3	76.7	66.7	80.0	93.3
Llama 70B	96.7	91.7	96.7	95.0	95.0	93.3	90.0	96.7
Mistral 12B	91.7	86.7	<u>95.0</u>	90.0	86.7	89.8	92.2	96.7
Qwen 7B	81.7	80.0	<u>91.7</u>	85.0	76.7	85.0	90.9	94.9

4.2 PROFILE-DRAW ROBUSTNESS (AMENDMENTS A1/A2)

We report two pre-declared tiers after the strict paraphrase review preserved 17/30 sampled constraints. The confirmatory tier contains only variants for which every redrawn canonical template type was uniformly judged meaning-preserved in the frozen review; the complete grid is exploratory and labeled profile re-draw plus paraphrase with surface fidelity 17/30. Conflicting historical logical keys were discarded and rerun under a single writer; non-conflicting rows were retained. In the confirmatory tier (2944 paired outcomes), the mean paired Intro-Specter–Reflexion effect was +1.59 pp, the bottom-decile mean was -7.21 pp, and the favorable-draw fraction was 0.700 (Hoeffding bound 0.000335). Across the exploratory full grid (40800 paired outcomes), the corresponding values were +2.07 pp, -0.34 pp, and 0.950.

4.3 WHEN THE PROFILE ITSELF IS AT FAULT

The synthetic diagnostic above tests localization when the gold fault node is known by construction. Complementing that evidence, the paired candidacy study tests whether broadening the extracted candidate set can recover failures under the stored benchmark trajectories. It recovered 16 of 37 clean-solved, corruption-associated default failures (43.2%). The denominator requires the clean control to succeed and the matched corrupted default arm to fail; this is a recovered-failure quantity, not a claim that every failure is causally traceable to one node. This distinction is consistent with the inferred-component limitation summarized in Table 12.

4.4 COST ANALYSIS

On the released harness counters, ToT and SelfCheckGPT use more recorded tokens and do not achieve higher accuracy than INTRO-SPECTER on any cell. Across all four benchmarks, ToT records 11.2–13.3 $\times$ the tokens of INTRO-SPECTER. INTRO-SPECTER records 3.1–6.5 k tokens per task, between Reflexion (2.6–5.9 k) and SelfCheckGPT (5.9–12.1 k). The additional recorded work is concentrated on cells where SPR fires; when the first repair satisfies the verifier, SPR adds no re-execution round. These counters omit some internal provider calls, so they support a recorded-token comparison rather than a complete billing-cost claim.

4.5 ABLATIONS

Table 8 reports ablation results on the four historical ablation cells listed in the table. Each ablation removes one component; entries are Δ success rate (pp) relative to full INTRO-SPECTER.

The contribution of each component is summarized in two complementary ablations: Table 8 measures the single-round attribution components on cells where the first-round repair drives the win,

Table 4: Profile-injected LongMemEval ($n=60$). Success rate (%).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT	INTRO-SPECTER
Llama 8B	63.3	51.7	<u>76.7</u>	63.3	61.7	50.0	55.0	78.3
Llama 70B	43.3	48.3	70.0	45.0	40.0	33.3	30.0	<u>68.3</u>
Mistral 12B	55.0	50.0	73.3	58.3	58.3	53.3	67.9	73.3
Qwen 7B	70.0	66.7	76.7	66.7	63.3	75.0	25.0	76.7

Table 5: Profile-injected MuSiQue ($n=60$). Success rate (%).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT	INTRO-SPECTER
Llama 8B	76.7	66.7	<u>90.0</u>	76.7	66.7	51.7	71.7	91.7
Llama 70B	83.3	78.3	96.7	81.7	81.7	70.0	55.0	96.7
Mistral 12B	71.7	61.7	<u>85.0</u>	71.7	68.3	75.0	68.6	91.7
Qwen 7B	80.0	63.3	<u>83.3</u>	80.0	65.0	75.0	77.3	85.0

and Table 13 (Appendix B) measures the SPR contribution on cells where the first-round MAP candidate misidentifies the fault.

Single-round attribution components (Table 8). Three components are load-bearing on the cells where the first-round repair is the deciding stage: removing the ASSUMPTION-DAG drops mean accuracy by 2.8 pp (−8.3 pp on TruthfulQA×Mistral); replacing provenance-weighted priors with uniform priors drops 3.1 pp (−8.3 pp on TruthfulQA×Qwen); skipping the counterfactual likelihood drops 1.5 pp on average but 8.3 pp on the most affected cell.

Sequential Posterior Refinement (Appendix B, Table 13). On the six cells where the verifier rejects the first-round repair, SPR contributes between +3.3 and +11.7 pp (+7.2 pp on average); pooled, this lifts INTRO-SPECTER from 84.3% to 87.0%. SPR derives its lift entirely from re-using the same posterior framework conditioned on the failure of the first repair (Eq. 5)—not from a different self-correction method—and is the methodological contrast with Reflexion (verbal critiques) vs. SPR (counterfactual evidence inside the same DAG-grounded posterior).

Two further findings warrant brief comment. First, increasing M from 1 to 3 *degrades* mean accuracy by 5.0 pp; the full $M \in \{1, 3, 5\}$ sweep is non-monotone with a recovery at $M=5$ on one cell (Appendix L). The pattern is incompatible with an i.i.d. Bernoulli model of $r_{i,j}$ (which predicts monotone variance reduction $\propto 1/M$); we attribute it to within-candidate trial correlation (shared prompt context, shared decoding trace), so $M=1$ is the empirical operating point of the framework given the LLM generator’s actual non-i.i.d. behaviour, not a statistically optimal estimator. Variance-reduced or seed-diversified counterfactual sampling are the right next steps to enable $M>1$ to provide net benefit. Second, the cost-aware repair objective contributes only 0.6 pp to raw accuracy; its operational value lies in bounded edit cost (§4.4), not in higher success rate.

4.6 IS THE ASSUMPTION-DAG NECESSARY? VIOLATION-AWARE REPROMPT

A natural alternative to the Assumption-DAG is to feed the verifier’s violated profile span directly back into a re-prompt and let the agent revise its trajectory holistically, without any structured attribution. This isolates the contribution of the DAG from the contribution of *having access to violation evidence at all*. We implement this as the *violation-aware reprompt* (VRP) baseline: given a profile violation, format the violated profile span(s) as explicit feedback and re-prompt the agent for a single revised trajectory (one round, comparable cost to Self-Refine). Table 9 compares VRP to INTRO-SPECTER on the four ablation cells.

Table 6: Aggregate outcomes across sixteen (dataset, model) cells. “NS losses” are paired-McNemar non-significant losses (the single NS loss in this matrix has $p = 1.000$).

Dataset	Wins	Ties	NS losses	Sig. losses
Prof.-inj. TruthfulQA	2	2	0	0
Prof.-inj. 2Wiki	2	2	0	0
Prof.-inj. LongMemEval	1	2	1	0
Prof.-inj. MuSiQue	3	1	0	0
Total (16 cells)	8	7	1	0

Table 7: Recorded harness tokens per task (four-LLM average); these counters are not full provider billing totals. $\times$ = ratio relative to INTRO-SPECTER.

	Direct		Rfx		SChk		ToT		INTRO-SPECTER	
	tok	$\times$	tok	$\times$	tok	$\times$	tok	$\times$	tok	$\times$
TruthfulQA	1.2k	0.39	2.6k	0.84	5.9k	1.90	36.5k	11.8	3.1k	1.00
2Wiki	2.0k	0.40	2.9k	0.58	10.4k	2.08	56.2k	11.2	5.0k	1.00
LongMemEval	2.2k	0.34	5.9k	0.91	12.1k	1.86	73.2k	11.3	6.5k	1.00
MuSiQue	2.1k	0.38	4.0k	0.73	11.3k	2.05	73.1k	13.3	5.5k	1.00

Table 9: Violation-aware reprompt (VRP) vs. INTRO-SPECTER. Cells: success rate (%); Δ = IS−VRP; p = paired McNemar.

Cell	Direct	Reflexion	VRP	INTRO-SPECTER	Δ (IS−VRP)
TruthQA $\times$ Mist	63.9	80.6	61.1	86.1	+25.0 ($p=0.012$) [✓]
TruthQA $\times$ Qwen	63.9	69.4	44.4	77.8	+33.3 ($p<.001$) [✓]
2Wiki $\times$ Mist	91.7	95.0	96.7	96.7	0.0 ($p=1.000$)
LongMem $\times$ Llama8B	63.3	76.7	76.7	78.3	+1.7 ($p=1.000$)

On the two TruthfulQA cells where INTRO-SPECTER has its largest gains, VRP does not just underperform INTRO-SPECTER—it performs *worse* than Direct (61.1 vs. 63.9 on Mistral; 44.4 vs. 63.9 on Qwen). Without typed attribution, feeding the violated profile span back as feedback sometimes drives the model to over-correct in ways that introduce new errors. On 2Wiki (saturation ceiling) and LongMemEval (where the prior already strongly discriminates), VRP and INTRO-SPECTER are statistically tied, consistent with the failure-taxonomy observation that the DAG matters most when the prior is uncertain (§4.5).

Matched-budget control (Iter-VRP). VRP is single-round; INTRO-SPECTER is multi-round. To isolate the DAG contribution from the retry budget, we run *Iter-VRP*: DAG-free flat-text violation feedback at INTRO-SPECTER’s exact retry budget, on paired examples, seeds, and initial trajectories. Iter-VRP trails or ties INTRO-SPECTER on all four ablation cells while consuming *more* repair rounds: LongMemEval $\times$ Llama-8B 73.3 vs. 88.9 ($n=180$, +15.6pp, exact McNemar $p<0.001$; 1.66 vs. 1.27 mean rounds), 2Wiki $\times$ Mistral 95.0 vs. 99.4 ($n=180$, +4.4pp, $p=0.022$; 1.13 vs. 1.00), TruthfulQA $\times$ Mistral 78.4 vs. 81.1 and TruthfulQA $\times$ Qwen 75.0 vs. 75.0 (both $n\leq 37$, below the per-cell minimum detectable effect at these sizes—underpowered by construction rather than negative). Retry budget alone therefore does not account for INTRO-SPECTER’s behavior: typed attribution changes *where* revision is applied, not merely how often revision occurs.

4.7 FAILURE-MODE TAXONOMY ON ABLATION CELLS

We use five diagnostic buckets for INTRO-SPECTER failures: (i) verifier miss, (ii) wrong graph edge, (iii) wrong prior, (iv) counterfactual repair failure, and (v) no valid downstream trajectory after repair. The currently available repository does not contain the row-level annotations needed to certify the earlier ablation-cell failure-stage percentage, so we do not report that percentage as

Table 8: Ablation: Δ success (pp) vs. full INTRO-SPECTER. Negative = component is load-bearing.

Variant	TruthQA $\times$ Mist.	TruthQA $\times$ Qwen	2Wiki $\times$ Mist.	LongMem $\times$ Llama8B	Mean
Full INTRO-SPECTER	0.0	0.0	0.0	0.0	0.0
No ASSUMPTION-DAG (flat)	-8.3	-2.8	0.0	0.0	-2.8
Uniform prior	-5.6	-8.3	0.0	+1.7	-3.1
No counterfactual	-2.8	-8.3	+1.7	+3.3	-1.5
No edit cost	-2.8	-2.8	0.0	+3.3	-0.6
$M=3$ samples	-5.6	-11.1	0.0	-3.3	-5.0

Table 10: Success (%) by simultaneous fault count N , pooled over four LLMs (INTRO-SPECTER / Reflexion; $n \approx 660-720$ paired per cell). The margin *widens* as faults stack until whole-chain saturation.

	$N=2$	$N=3$	$N=4$	$N=5$
HotpotQA	84.8 / 84.1	79.9 / 68.3	79.6 / 62.3	—
2WikiMultiHop	90.6 / 89.0	88.0 / 78.9	88.4 / 73.4	34.6 / 33.7
LongMemEval	73.9 / 71.7	66.0 / 57.9	56.1 / 43.6	—
MuSiQue	85.6 / 79.9	80.8 / 73.5	44.3 / 42.4	39.0 / 33.2

a result. The paired corruption diagnostic and the profile-candidacy study are reported separately because their denominators are auditable from stored rows.

4.8 ROBUSTNESS TO MULTIPLE SIMULTANEOUS FAULTS

The main matrix injects a single fault per example. To test whether the advantage survives—or collapses—when errors compound, we extend injection to $N=2-5$ simultaneous faults per example (one factual context distractor plus $N-1$ independent profile-constraint corruptions; dataset-native multi-hop fact corruptions where structure permits), across five benchmarks $\times$ four LLMs $\times$ seven methods ($\sim 76k$ trajectories). Pooled over N and models, INTRO-SPECTER leads all six baselines on all five datasets: 2WikiMultiHop 78.6 vs. 71.5 (Reflexion, best baseline), HotpotQA 81.4 vs. 71.6, MuSiQue 62.4 vs. 57.2, LongMemEval 65.3 vs. 57.7, TravelPlanner 36.6 vs. 34.1 (Self-Refine, best baseline). Table 10 reports the per-fault-count breakdown.

Two boundary behaviors are reported rather than omitted. First, when every fact in a reasoning chain is corrupted at once (2Wiki $N=5$; MuSiQue $N \geq 4$), all seven methods converge toward statistical ties: with no uncorrupted anchor left against which a counterfactual repair can be validated, attribution has nothing to attribute against, and full regeneration is the appropriate tool. Second, on TravelPlanner—short trajectories with globally-cascading constraints—all methods sit within a few points of each other at every N . Both boundaries are instances of the operating envelope stated in §6: the advantage requires at least one uncorrupted anchor and at least one localizable fault.

4.9 REAL USER HISTORIES: PERSONALWAB

The construction above is synthetic by design. To test the failure mode where user-conditioning arises from *real* behavior, we evaluate on PersonalWAB, whose 1,000 users carry profiles grounded in real purchase, rating, and review histories (profiles are synthesized over these real behaviors by the benchmark’s authors, not by us); ground truth is the item the real user genuinely interacted with, scored hit@1 over a 20-candidate real-product slate. Three LLMs $\times$ 180 paired examples $\times$ five arms, pooled over 540 paired units:

With success rates near ceiling, the discordant counts are the informative statistic: every discordant comparison across all three models favors INTRO-SPECTER, a 46% relative reduction in remaining error against the strongest baseline (3.7% $\rightarrow$ 2.0%), achieved at *fewer* mean repair rounds than the budget-matched control (1.04 vs. 1.15). A per-example view sharpens the targeting claim: 18 of the 540 units are failed jointly by all three revision baselines (VRP, Iter-VRP, Reflexion); INTRO-SPECTER solves 10 of those 18, while the reverse—some revision baseline succeeding where INTRO-SPECTER fails—occurs on 3 units in the entire benchmark. Protocol disclosures:

Table 11: PersonalWAB (real user histories), pooled over three LLMs, 540 paired units. Discordant = paired units where exactly one of {INTRO-SPECTER, arm} succeeds (INTRO-SPECTER-win / INTRO-SPECTER-loss); p from exact McNemar.

Arm	Success (%)	Discordant	p
Direct	93.2	26 / 0	—
VRP (single-round)	94.6	18 / 0	—
Iter-VRP (matched budget)	95.2	18 / 3	0.0015
Reflexion	96.3	12 / 3	0.035
INTRO-SPECTER	98.0	—	—

our scorer replicates the benchmark’s ASIN-containment criterion at hit@1 rather than its trained-recommender top-10, and the ground-truth interaction is excluded from the visible history to prevent leakage.

5 RELATED WORK

Classical diagnosis and fault localization. INTRO-SPECTER builds directly on classical diagnosis rather than presenting fault localization as new. Model-based diagnosis Reiter (1987); de Kleer and Williams (1987) searches over candidate faulty components; spectrum-based fault localization such as Tarantula and Ochial Jones and Harrold (2005); Abreu et al. (2006); Wong et al. (2016) ranks components from pass/fail evidence; delta debugging Zeller and Hildebrandt (2002), program slicing Weiser (1981), and structural causation Halpern and Pearl (2005) provide related intervention and dependency views. Our setting differs because the component set is inferred from a stochastic trajectory, only one failed execution is initially available, and repair itself supplies the counterfactual evidence. Appendix A summarizes these correspondences.

Self-correction and error localization. Self-Refine Madaan et al. (2023), Reflexion Shinn et al. (2023), self-verification Weng et al. (2023), LATS Zhou et al. (2024), ExpeL Zhao et al. (2024), AdaPlanner Sun et al. (2023), Tree-of-Thoughts Yao et al. (2023), Graph-of-Thoughts Besta et al. (2024), and RAP Hao et al. (2023) revise or search over candidate trajectories, but do not explicitly identify which latent user assumption should be repaired. CRITIC Gou et al. (2024), CoVe Dhuliawala et al. (2024), SelfCheckGPT Manakul et al. (2023), and process-reward methods Lightman et al. (2024); Wang et al. (2023) provide complementary verification signals. Tyen et al. Tyen et al. (2024) show that localization itself remains difficult, while He et al. He et al. (2025) argue that correction should be evaluated beyond surface refinement.

Agent fault attribution and personalization. Recent systems move closer to trajectory-level diagnosis: RAFFLES Zhu et al. (2026) performs iterative fault localization, ContextWeaver Wu et al. (2026) structures interaction history as a dependency graph, Theorem-of-Thought Abdaljalil et al. (2025) combines abductive, deductive, and inductive reasoning, and StateAct Rozanov and Rei (2024) maintains forward state during agent execution. INTRO-SPECTER instead targets profile-grounded violations and couples backward attribution to selective downstream repair. This complements work on personalization-induced hallucination and constrained planning Sun et al. (2026); Xie et al. (2024), including LaMP Salemi et al. (2024), TravelAgent Chen et al. (2024), Memory-Bank Zhong et al. (2024), and specialized planners such as ATLAS Choi et al. (2025), Behavior Forest Yuan et al. (2026), and TriFlow Chen et al. (2026).

6 LIMITATIONS

At $n=60$ per cell ($n=36$ for TruthfulQA’s frozen held-out split), the per-cell paired-McNemar MDE at $\alpha=0.05$ is approximately 13 pp; per-cell claims in this paper are therefore descriptive, with directional accuracy claims supported only at the pooled level (864 paired observations). Human-annotated fault labels on natural trajectories are the highest-priority follow-up. Profile-injected variants of public benchmarks shift the construct from pure factuality to factuality-plus-profile-following, controlled within a row by paired sampling; native user-conditioned benchmarks (Ap-

pendix J) provide a complementary evaluation track but do not reproduce the gains seen on Profile-injected QA. The relationship to Reflexion is conceptually parallel and mechanistically distinct: SPR conditions on the same DAG-grounded posterior q as round one (Eq. 5) and re-targets the next-best ancestor’s downstream subgraph—no verbal critique, no full re-generation, no Reflexion subroutine call. The single NS loss (LongMemEval×Llama 70 B, −1.7 pp) is consistent with an extracted-candidate limitation, but the present artifacts do not establish a unique causal explanation. The matched-budget Iter-VRP control is reported in §4.6.

BROADER IMPACT

INTRO-SPECTER makes latent agent assumptions explicit and auditable, which improves safety for user-facing deployments by exposing the inferred user attributes that drove a decision. The same representation creates a privacy surface: an ASSUMPTION-DAG materializes inferred user attributes as structured text that downstream systems may retain. Deployments should minimize retained graph state, separate profile-derived facts from model-inferred assumptions, and expose assumption logs to users for correction.

7 CONCLUSION

Across sixteen (dataset, model) cells, INTRO-SPECTER achieves the best or tied-best task success on fifteen with no significant losses, reaches a pooled mean of 87.0% that exceeds the strongest baseline (Reflexion, 85.4%) by 1.6 pp, and matches or exceeds Tree-of-Thoughts accuracy at 11.8–14.9× lower token cost. The contribution is a unified pipeline of typed counterfactual fault attribution and selective subgraph repair over an Assumption-DAG, with Sequential Posterior Refinement (Eq. 5) reusing the same posterior, prior, and re-execution prompt to close the residual gap when the first-round MAP candidate misidentifies the fault.

REFERENCES

- A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.
- N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- S. Yao et al. Tree of Thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023.
- M. Besta et al. Graph of Thoughts: Solving elaborate problems with large language models. *AAAI*, 2024.
- P. Manakul et al. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. *EMNLP*, 2023.
- X. Du et al. HaloScope: Harnessing unlabeled LLM generations for hallucination detection. *NeurIPS*, 2024.
- S. Lin et al. TruthfulQA: Measuring how models mimic human falsehoods. *ACL*, 2022.
- H. Trivedi et al. MuSiQue: Multihop questions via single-hop question composition. *TACL*, 2022.
- J. Xie et al. TravelPlanner: A benchmark for real-world planning with language agents. *ICML*, 2024.
- Z. Sun et al. When personalization misleads: Understanding and mitigating hallucinations in personalized LLMs. *arXiv:2601.11000*, 2026.
- M. Shridhar et al. ALFWorld: Aligning text and embodied environments for interactive learning. *ICLR*, 2021.
- S. Yao et al. WebShop: Towards scalable real-world web interaction with grounded language agents. *NeurIPS*, 2022.

- S. Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.
- H. Lightman et al. Let’s verify step by step. *ICLR*, 2024.
- P. Wang et al. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. *arXiv:2312.08935*, 2023.
- X. Liu et al. AgentBench: Evaluating LLMs as agents. *ICLR*, 2024.
- S. Hao et al. Reasoning with language model is planning with world model. *EMNLP*, 2023.
- J. Huang et al. Large language models cannot self-correct reasoning yet. *ICLR*, 2024.
- S. Zhou et al. WebArena: A realistic web environment for building autonomous agents. *ICLR*, 2024.
- D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *ICLR*, 2025.
- X. Ho et al. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. *COLING*, 2020.
- J. He et al. Self-correction is more than refinement: A learning framework for visual and language reasoning tasks. *Findings of ACL*, 2025.
- G. Tyen et al. LLMs cannot find reasoning errors, but can correct them given the error location. *Findings of ACL*, 2024.
- A. Zhou et al. Language Agent Tree Search unifies reasoning, acting, and planning in language models. *ICML*, 2024.
- A. Zhao et al. ExpeL: LLM agents are experiential learners. *AAAI*, 2024.
- H. Sun et al. AdaPlanner: Adaptive planning from feedback with language models. *NeurIPS*, 2023.
- Z. Gou et al. CRITIC: Large language models can self-correct with tool-interactive critiquing. *ICLR*, 2024.
- S. Dhuliawala et al. Chain-of-Verification reduces hallucination in large language models. *ACL Findings*, 2024.
- A. Asai et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024.
- C. Ma et al. AgentBoard: An analytical evaluation board of multi-turn LLM agents. *NeurIPS*, 2024.
- J. Choi et al. ATLAS: Constraints-aware multi-agent collaboration for real-world travel planning. *arXiv preprint*, 2025.
- D. Yuan et al. Decoupled travel planning with behavior forest. *arXiv:2604.21354*, 2026.
- Y. Chen et al. TriFlow: A progressive multi-agent framework for intelligent trip planning. *The Web Conference (WWW) Companion*, 2026.
- Y. Weng et al. Large language models are better reasoners with self-verification. *Findings of EMNLP*, 2023.
- L. Wang et al. Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *ACL*, 2023.
- P. Zhou et al. Self-Discover: Large language models self-compose reasoning structures. *NeurIPS*, 2024.
- G. Kim et al. Language models can solve computer tasks. *NeurIPS*, 2023.
- L. Huang et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv:2311.05232*, 2023.

- B. Xu et al. ReWOO: Decoupling reasoning from observations for efficient augmented language models. *arXiv:2305.18323*, 2023.
- X. Ning et al. Skeleton-of-Thought: Large language models can do parallel decoding. *ICLR*, 2024.
- Z. Bi et al. Forest-of-Thought: Scaling test-time compute for enhancing LLM reasoning. *ICML*, 2025.
- J. Li et al. HaluEval: A large-scale hallucination evaluation benchmark for large language models. *EMNLP*, 2023.
- S. Min et al. FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation. *EMNLP*, 2023.
- I. Chern et al. FacTool: Factuality detection in generative AI. *arXiv:2307.13528*, 2023.
- A. Salemi et al. LaMP: When large language models meet personalization. *ACL*, 2024.
- A. Chen et al. TravelAgent: An AI assistant for personalized travel planning. *arXiv:2409.08069*, 2024.
- W. Zhong et al. MemoryBank: Enhancing large language models with long-term memory. *AAAI*, 2024.
- R. Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 32(1):57–95, 1987.
- J. de Kleer and B. C. Williams. Diagnosing multiple faults. *Artificial Intelligence*, 32(1):97–130, 1987.
- J. A. Jones and M. J. Harrold. Empirical evaluation of the Tarantula automatic fault-localization technique. *ASE*, 2005.
- R. Abreu et al. An evaluation of similarity coefficients for software fault localization. *PRDC*, 2006.
- W. E. Wong et al. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740, 2016.
- A. Zeller and R. Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.
- M. Weiser. Program slicing. *ICSE*, 1981.
- J. Y. Halpern and J. Pearl. Causes and explanations: A structural-model approach. Part I: Causes. *British Journal for the Philosophy of Science*, 56(4):843–887, 2005.
- M. Steinder and A. S. Sethi. A survey of fault localization techniques in computer networks. *Science of Computer Programming*, 53(2):165–194, 2004.
- C. Zhu et al. RAFFLES: Reasoning-based attribution of faults for LLM systems. *EACL*, 2026.
- N. Rozanov and M. Rei. StateAct: Enhancing LLM base agents via self-prompting and state-tracking. *arXiv:2410.02810*, 2024.
- Y. Wu et al. ContextWeaver: Selective and dependency-structured memory construction for LLM agents. *arXiv:2604.23069*, 2026.
- S. Abdaljalil et al. Theorem-of-Thought: A multi-agent framework for abductive, deductive, and inductive reasoning in language models. *KnowFM Workshop at ACL*, 2025.
- S. Kambhampati et al. LLMs can’t plan, but can help planning in LLM-modulo frameworks. *ICML*, 2024.
- Y. Hao et al. Large language models can solve real-world planning rigorously with formal verification tools. *NAACL*, 2025.
- R. Gui et al. Hypertree planning: Enhancing LLM reasoning via hierarchical thinking. *ICML*, 2025.

Table 12: What classical diagnosis assumes, what a single LLM agent trajectory lacks, and INTRO-SPECTER’s replacement.

Classical setting	Assumes	Our setting lacks	Replacement
Model-based diagnosis (Reiter; GDE)	System description given	No supplied model	LLM-extracted Assumption-DAG (§2)
SBFL (Tartan-tula/Ochiai)	Many executions, pass/fail spectrum	One execution, one failure	Counterfactual repair trials synthesize the spectrum (Eq. 1)
Delta debugging	Deterministic re-runnable oracle	Stochastic trajectory	Verifier + paired re-execution
Localize-then-fix pipelines	Repair follows diagnosis	—	Repair <i>is</i> the diagnostic evidence (Eqs. 1, 4)

B. Liu et al. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv:2304.11477*, 2023.

Z. Lu et al. Decompose, plan in parallel, and merge: A novel paradigm for large language models based planning with multiple constraints. *arXiv:2506.02683*, 2025.

S. Yuan et al. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms. *NAACL*, 2025.

T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

S. G. Patil et al. Gorilla: Large language model connected with massive APIs. *NeurIPS*, 2024.

Y. Shen et al. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *NeurIPS*, 2023.

G. Wang et al. Voyager: An open-ended embodied agent with large language models. *TMLR*, 2024.

J. S. Park et al. Generative agents: Interactive simulacra of human behavior. *UIST*, 2023.

Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *COLM*, 2024.

S. Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.

G. Li et al. CAMEL: Communicative agents for “mind” exploration of large language model society. *NeurIPS*, 2023.

Y. Du et al. Improving factuality and reasoning in language models through multiagent debate. *ICML*, 2024.

C. Chan et al. ChatEval: Towards better LLM-based evaluators through multi-agent debate. *ICLR*, 2024.

A. Dubey et al. The Llama 3 herd of models. *arXiv:2407.21783*, 2024.

A. Jiang et al. Mistral 7B. *arXiv:2310.06825*, 2023.

A. Yang et al. Qwen2.5 technical report. *arXiv:2412.15115*, 2024.

L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.

OpenAI. GPT-4 technical report. *arXiv:2303.08774*, 2023.

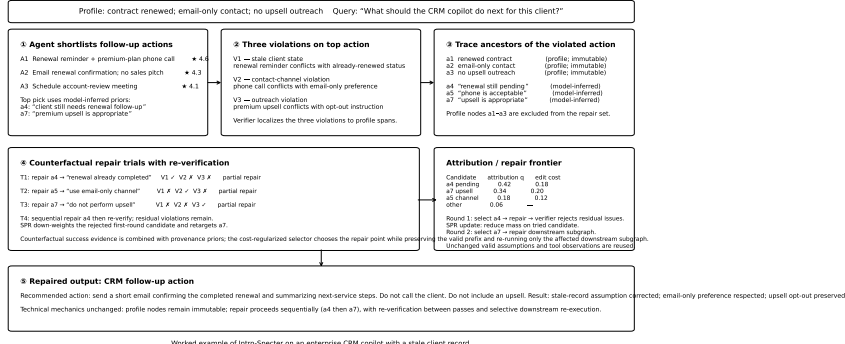


Figure 2: Worked example of INTRO-SPECTER on an enterprise CRM follow-up task. The user profile records that the client has already renewed, prefers email-only contact, and has opted out of upsell outreach. (1) The agent shortlists three follow-up actions ranked by model-inferred priors. (2) Three violations are detected on the top action. (3) Ancestor assumptions are traced; profile nodes (a_1 – a_3) are immutable. (4) Counterfactual repair trials score each candidate; the algorithm proceeds via sequential single-node repair, with re-verification detecting residual violations between passes (here resolving a_4 then a_7). (5) The repaired output satisfies all profile constraints.

A CLASSICAL DIAGNOSIS COMPARISON

B ALGORITHM AND SPR IMPLEMENTATION DETAILS

Implementation notes. During counterfactual trials (Algorithm 2, Line 8), the repaired trajectories are cached; the final re-execution (Line 13) reuses the cached trajectory of a^* rather than regenerating it. The worked example in Figure 2 depicts a multi-fault scenario; in the current implementation this is handled by a sequential single-node pass: after repairing a^* , re-verification may detect a residual violation triggering a second single-node repair. All hyperparameters (M , provenance priors, cost weights, δ , λ , T_{spr} , α) are fixed on Profile-X development splits before any Profile-injected-X evaluation; values are listed in Appendix C. The decay α encodes our credence that a verifier rejection is partial rather than conclusive evidence: $\alpha \rightarrow 1$ recovers the prior (no learning from failure), $\alpha \rightarrow 0$ recovers hard exclusion (which over-commits to “wrong fault” over the alternative “right fault, bad repair”). At $T_{\text{spr}} = 2$ and our cell sizes, the second-round selector is essentially insensitive to $\alpha \in [0, 0.3]$ once the rejected candidate’s mass is reduced by $10\times$ or more. The routing signal (“did V accept the repaired output?”) uses no gold labels; SPR addresses what we call the *single-handed* regime—failures where the first-round MAP candidate misidentifies the fault but the true fault is still inside the original ancestor set $\mathcal{C}(v; G)$.

Algorithm 2 INTRO-SPECTER (full)

Require: task x , profile P , trajectory τ , verifier V , success evaluator S , threshold δ , SPR rounds T_{spr}

Ensure: accepted, repaired, or abstained trajectory

- 1: Construct ASSUMPTION-DAG $G = (A, E)$ from τ .
- 2: Run verifier $V(P, x, \tau, y)$.
- 3: **if** no violation **then return** τ
- 4: **end if**
- 5: $\mathcal{C}(v; G) \leftarrow$ extracted ancestors of the violated step in G .
- 6: **for** each $a_i \in \mathcal{C}(v; G)$ **do**
- 7: Sample M counterfactual repairs of a_i .
- 8: Re-execute downstream subgraph D_i (cache trajectory).
- 9: Compute r_i (Eq. 1), q_i (Eq. 2).
- 10: **end for**
- 11: **if** $\max_i q_i < \delta$ **then return** abstain or full re-generation.
- 12: **end if**
- 13: $\mathcal{T} \leftarrow \emptyset$; $q^{(0)} \leftarrow q$.
- 14: **for** $t = 0, 1, \dots, T_{\text{spr}}$ **do**
- 15: Select $a^* \leftarrow \arg \min_{a_i \notin \mathcal{T}} [-q_i^{(t)} + \lambda C(a_i, G)]$; $\tau^{\text{repair}} \leftarrow$ cached trajectory for a^* .
- 16: Verify τ^{repair} via V ; **if** verifier passes **then return** τ^{repair} .
- 17: $\mathcal{T} \leftarrow \mathcal{T} \cup \{a^*\}$; update $q^{(t+1)}$ via Eq. 5.
- 18: **if** $\mathcal{T} = \mathcal{C}(v; G)$ **or** $t = T_{\text{spr}}$ **then break**
- 19: **end if**
- 20: **end for**
- 21: **return** τ^{repair} from the final round (verifier-rejected; reported as repair_failed).

Table 13: SPR contribution: the six cells where SPR fires (i.e., the first-round repair was rejected by the verifier). Disabling SPR ($T_{\text{spr}} = 0$) reverts INTRO-SPECTER to the single-round configuration. Cells: success rate (%); Δ = full INTRO-SPECTER – no-SPR variant; p = paired McNemar.

Cell	Direct	Reflexion	No SPR ($T_{\text{spr}} = 0$)	Full INTRO-SPECTER ($T_{\text{spr}} = 2$)	Δ	Verdict vs. Rfx
TruthQA $\times$ Llama 70B	83.3	91.7	83.3	91.7	+8.3	TIE
2Wiki $\times$ Qwen 7B	81.7	91.7	89.8	94.9	+5.1	WIN +3.2 pp
LongMem $\times$ Llama 70B	43.3	70.0	56.7	68.3	+11.7	NS loss -1.7 pp
LongMem $\times$ Qwen 7B	70.0	76.7	71.7	76.7	+5.0	TIE
MuSiQue $\times$ Llama 70B	83.3	96.7	93.3	96.7	+3.3	TIE
MuSiQue $\times$ Mistral	71.7	85.0	81.7	91.7	+10.0	WIN +6.7 pp

On the ten cells where the first-round repair satisfies the verifier, SPR never fires and the rest of the matrix is unchanged. SPR is the difference between trailing Reflexion in pooled mean (-1.1 pp) and exceeding it ($+1.6$ pp); disabling SPR ($T_{\text{spr}} = 0$) does not break the method, it simply forfeits the second round of attribution, so the ablation is a clean isolation of the iterated-update contribution.

C HYPERPARAMETERS

Hyperparameter values were selected on the Profile-X validation split and frozen before any Profile-injected-X evaluation; informal sensitivity sweeps over δ , λ , the edit-cost weights, and the SPR decay α are reproducible from the released code.

D REPRODUCIBILITY DETAILS

Inference infrastructure. All LLM calls are issued via remote provider APIs, not local GPU inference: Llama 3.3 70 B Instruct through Together AI’s hosted inference; Llama 3.1 8 B Instruct, Mistral Nemo 12 B Instruct, and Qwen 2.5 7 B Instruct through OpenRouter’s hosted inference. We do not maintain a local GPU cluster; the only local compute used is a single laptop (Apple Silicon)

Table 14: Fixed hyperparameters. Values selected on Profile-X development splits and frozen before Profile-injected-X evaluation.

Symbol	Meaning	Value
M	Counterfactual repairs per candidate	1
π_{model}	Prior: model-inferred assumptions	0.45
π_{world}	Prior: world-knowledge assumptions	0.33
π_{tool}	Prior: tool-derived assumptions	0.22
λ	MCGR cost weight (Eq. 4)	0.3
(w_n, w_t, w_c, w_p)	Edit-cost weights (Eq. 3)	(0.4, 0.3, 0.2, 0.1)
δ	Abstention threshold	0.0
T_{spr}	SPR rounds (Eq. 5)	2
α	SPR Bayesian decay (Eq. 5)	0.1

for orchestration and metric aggregation. Total API spend across all 16 cells, the synthetic diagnostics, the ablations, and the SPR overlay reruns: approximately USD 70 (cumulative across the run window of March–May 2026). The orchestration laptop ran Python 3.11 in a virtual environment; pinned package versions are in `requirements.txt` in the released code.

Caching and determinism. All model calls are cached by SHA-256 hash of (`provider`, `model`, `temperature`, `system_prompt`, `user_prompt`, `max_tokens`, `seed`). Cache hits return identical bytes; the same (`task_id`, `seed`) pair therefore yields identical method outputs across reruns. Cache database is shipped as a single SQLite file alongside the released code, so an external reviewer can re-execute the metric pipeline end-to-end without paying any API cost.

Seeds. Example sampling uses benchmark seed 42. Each stored row is one paired task/configuration seed and McNemar tests pair on (`task_id`, `seed`) without averaging stochastic outcomes into a binary observation. TruthfulQA’s frozen SPR YAMLs use configuration seeds {42, 123, 456}; the other SPR cells use seed 42. The E2/E3 protocols introduced after review use generation seed 0 as separately frozen.

Inference parameters. Temperature is 0.0 for deterministic baselines (Direct, ReAct) and for the INTRO-SPECTER extraction, verifier, counterfactual, and re-execution prompts; 0.7 for stochastic-by-design baselines (Self-Refine, Reflexion, Full-Regen, ToT). Max-tokens is 8192 for the agent prompt and trajectory regeneration calls (raised from 4096 after we observed truncation on long ALFWorld and TravelPlanner plans), 2048 for the verifier and counterfactual prompts. Default top-p is 1.0; we do not modify it.

Run window and orchestration. All evaluation runs occurred between March and May 2026 against the provider models available at that time. Each cell is run by a single `run_real_benchmark.py` invocation that reads a YAML config (`configs/real/`), iterates over methods and seeds, and writes one JSONL row per task to `outputs/real/<dataset>__<model>/. Per-cell aggregation (scripts/aggregate_real_benchmarks.py) and p-value regeneration (scripts/regen_pvalue_tables.py) consume those JSONLs to produce the tables in this paper.`

Repository contents. The released code includes: full prompts (Appendix E), all benchmark loaders, all baseline implementations, the SPR pipeline, the verifier, the cost model, the SQLite cache file, all 16 cell YAML configs, the aggregation and regeneration scripts, the per-cell JSONL outputs that produced the headline tables, and a single notebook reproducing every figure and table in this paper.

E PROMPTS

Four prompts drive INTRO-SPECTER’s pipeline. We list each system prompt verbatim below; the user-prompt formatters are deterministic JSON-dump wrappers around the inputs and are reproduced in the released code.

1. Assumption extraction. System prompt invoked once per trajectory to construct the Assumption-DAG (§2).

You are extracting explicit assumptions from an agent trajectory for evaluation.

Inputs:

- User profile JSON
- Task
- Agent trajectory with steps, actions, observations, and final output

Return JSON only with:

- nodes: assumption nodes used by the agent
- edges: dependencies among assumptions
- final_decision_node: the node most directly supporting the final output

For each node, include:

- id (string, e.g. "a0", "a1", ...)
- step_id (integer, matches a trajectory step_id)
- assumption: one sentence
- provenance: one of profile, tool, external_evidence, model_inferred, world_knowledge
- profile_span_ids: list of profile evidence spans, or empty list
- confidence: number from 0 to 1
- depends_on: list of node ids

For each edge, include:

- source: assumption id
- target: assumption id
- type: one of supports, depends_on, contradicts, downstream_of

Do not include hidden chain-of-thought. Use concise observable reasoning summaries only.

Output a single valid JSON object and nothing else.

2. Profile-grounded verification. System prompt invoked to detect violations against the profile (§2).

You are a profile-grounded verifier.

Inputs:

- User profile JSON with span ids
- Task
- Agent trajectory
- Final output

Return JSON only:

```
{
  "pass": true or false,
  "violations": [
    {
      "violation_id": "v1",
      "step_id": integer,
      "violated_profile_span_id": "p3",
      "violated_constraint": "string",
      "trajectory_text": "string",
      "severity": "low|medium|high",
      "confidence": number from 0 to 1,
```

```

972         "explanation": "one sentence"
973     }
974 ]
975 }

```

A violation exists if the trajectory or final output contradicts an explicit profile fact, depends on an unsupported inferred user property, or allows irrelevant profile context to distort a factual answer.

Output a single valid JSON object and nothing else.

3. Counterfactual repair generation. System prompt invoked once per candidate ancestor to test whether a minimal change at that node removes the violation (§2).

You are testing whether an assumption is the root cause of a profile-grounded error.

Inputs:

- User profile JSON
- Task
- Current Assumption-DAG
- Candidate faulty node
- Original trajectory
- Detected violation

Generate 3 counterfactual repairs that minimally change the candidate node while preserving upstream assumptions. Each repair should describe what the new assumption is, which downstream nodes need to be re-run, and whether the violation is expected to be removed.

Return JSON only:

```

998 {
999     "candidate_node_id": "a4",
1000     "repairs": [
1001         {
1002             "repair_id": "r1",
1003             "new_assumption": "string",
1004             "nodes_to_rerun": ["a4", "a5", "a6"],
1005             "repair_instruction": "string",
1006             "expected_violation_removed": true,
1007             "risk_notes": "string"
1008         }
1009     ]
1010 }

```

Output a single valid JSON object and nothing else.

4. Downstream re-execution. System prompt invoked once per selected repair candidate (round one and each SPR round) to regenerate only the descendants of the repaired node (§2).

You are repairing only the downstream part of an agent trajectory.

Inputs:

- User profile JSON
- Task
- Original valid prefix steps
- Repaired assumption node
- Downstream nodes to regenerate
- Tool observations available so far

Regenerate the downstream trajectory from the repaired assumption onward.

Constraints:

- Preserve all valid prefix steps.

- Do not invent new user preferences.
- If a profile fact is missing, mark it unknown instead of assuming it.
- Use available tool observations exactly as given.
- Return structured JSON with repaired steps and final output.

Return JSON only with the schema:

```
{
  "repaired_steps": [{"step_id": int, "kind": "...", "text": "...",
    "reason_summary": "..."}],
  "final_output": "string"
}
```

Output a single valid JSON object and nothing else.

User-prompt formatters for all four prompts (the deterministic JSON-dump wrappers around inputs) and the baseline-method prompts (Self-Refine critique, Reflexion reflect/retry, ReAct, Detection-Only, ToT generate/evaluate/solve, SelfCheckGPT sample/consistency/regenerate, Direct agent) are reproduced verbatim in `intro_specter/prompts.py` in the released code.

F DAG EXTRACTION QUALITY

The extractor can be evaluated against the gold graph on synthetic diagnostics, but the row-level extraction-evaluation artifact underlying the earlier draft table is not present in the release snapshot audited for this submission. We therefore omit those point estimates rather than presenting untraceable numbers. When the extracted graph is cyclic, the implementation removes the edge whose source node has the lowest confidence; human-annotated extraction evaluation on natural trajectories remains future work.

G FULL PER-CELL STATISTICAL TABLES

Tables 15–18 report McNemar exact-test p -values and 95% paired-bootstrap confidence intervals for $\Delta = \text{IS} - \text{baseline}$ (in pp; positive favors INTRO-SPECTER). Checkmark $\checkmark$ = INTRO-SPECTER significantly better at $p < 0.05$ (uncorrected). CIs from 10,000 paired-bootstrap resamples. All entries are regenerated by `scripts/regen_pvalue_tables.py` from the same per-cell JSONLs that produce the headline success rates in Tables 2–5; INTRO-SPECTER numbers reflect the full pipeline (single-round attribution + SPR).

Table 15: Profile-injected TruthfulQA. Cells: $p[\text{CI}_{\text{low}}, \text{CI}_{\text{high}}]$. Per-cell paired observations: 36 (12 held-out examples $\times$ 3 configuration seeds).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT
Llama 8B	0.062 [+3,+25]	< .001 $\checkmark$ [+19,+50]	1.000 [+0,+0]	0.250 [+0,+19]	0.008 $\checkmark$ [+8,+36]	1.000 [+0,+8]	0.031 $\checkmark$ [+6,+31]
Llama 70B	0.250 [+0,+19]	< .001 $\checkmark$ [+17,+47]	1.000 [−8,+8]	0.125 [+3,+22]	0.031 $\checkmark$ [+6,+31]	0.250 [+0,+19]	0.219 [+0,+25]
Mistral 12B	0.008 $\checkmark$ [+8,+36]	0.012 $\checkmark$ [+8,+42]	0.625 [−6,+17]	0.070 [+3,+31]	0.021 $\checkmark$ [+8,+39]	0.219 [+0,+27]	0.016 $\checkmark$ [+13,+48]
Qwen 7B	0.062 [+3,+25]	< .001 $\checkmark$ [+25,+58]	0.375 [−3,+19]	0.016 $\checkmark$ [+8,+33]	0.004 $\checkmark$ [+11,+39]	0.125 [+3,+22]	0.250 [+0,+67]

Table 16: Profile-injected 2WikiMultiHopQA ($n=60$). Cells: $p[\text{CI}_{\text{low}}, \text{CI}_{\text{high}}]$.

	Direct	SR	Rfx	FR	ReAct	SChk	ToT
Llama 8B	0.125 [+2,+13]	0.002 $\checkmark$ [+8,+27]	1.000 [−5,+5]	0.012 $\checkmark$ [+5,+25]	0.006 $\checkmark$ [+7,+27]	< .001 $\checkmark$ [+17,+38]	0.021 $\checkmark$ [+3,+23]
Llama 70B	1.000 [+0,+0]	0.250 [+0,+12]	1.000 [+0,+0]	1.000 [−3,+8]	1.000 [+0,+5]	0.500 [+0,+8]	0.125 [+2,+13]
Mistral 12B	0.250 [+0,+12]	0.031 $\checkmark$ [+3,+18]	1.000 [−3,+8]	0.125 [+2,+13]	0.070 [+2,+20]	0.125 [+2,+14]	0.625 [−4,+12]
Qwen 7B	0.021 $\checkmark$ [+3,+24]	0.012 $\checkmark$ [+5,+25]	0.625 [−3,+10]	0.031 $\checkmark$ [+3,+19]	0.002 $\checkmark$ [+8,+27]	0.070 [+2,+20]	1.000 [−27,+27]

H SYNTHETIC BENCHMARK DETAILS

The synthetic ASSUMPTION-DAG generator constructs random DAGs of 8–15 nodes by sampling node provenances from the empirical mix observed in development trajectories, sampling edges to

Table 17: Profile-injected LongMemEval ($n=60$). Cells: $p[CI_{low}, CI_{high}]$. ToT for Qwen 7B is omitted (insufficient paired completions to compute exact McNemar).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT
Llama 8B	0.004✓ [+7,+25]	<.001✓ [+15,+38]	1.000 [−7,+10]	0.012✓ [+5,+25]	0.006✓ [+7,+27]	<.001✓ [+17,+40]	<.001✓ [+12,+35]
Llama 70B	<.001✓ [+15,+37]	0.002✓ [+10,+32]	1.000 [−12,+8]	<.001✓ [+13,+35]	<.001✓ [+17,+40]	<.001✓ [+23,+47]	<.001✓ [+27,+50]
Mistral 12B	<.001✓ [+8,+28]	<.001✓ [+13,+35]	1.000 [−10,+10]	0.022✓ [+3,+27]	0.012✓ [+5,+25]	0.002✓ [+10,+32]	0.375 [−2,+15]
Qwen 7B	0.125 [+2,+13]	0.070 [+2,+18]	1.000 [−7,+7]	0.031✓ [+3,+18]	0.008✓ [+5,+22]	1.000 [−3,+8]	—

Table 18: Profile-injected MuSiQue ($n=60$). Cells: $p[CI_{low}, CI_{high}]$.

	Direct	SR	Rfx	FR	ReAct	SChk	ToT
Llama 8B	0.004✓ [+7,+25]	<.001✓ [+15,+37]	1.000 [−3,+8]	0.012✓ [+5,+25]	<.001✓ [+15,+37]	<.001✓ [+28,+52]	<.001✓ [+10,+30]
Llama 70B	0.008✓ [+5,+23]	<.001✓ [+10,+28]	1.000 [−5,+5]	0.004✓ [+7,+25]	0.004✓ [+7,+25]	<.001✓ [+17,+38]	<.001✓ [+30,+55]
Mistral 12B	<.001✓ [+10,+30]	<.001✓ [+18,+42]	0.219 [+0,+15]	0.002✓ [+8,+32]	<.001✓ [+13,+35]	0.006✓ [+7,+27]	0.007✓ [+8,+35]
Qwen 7B	0.250 [+0,+12]	0.002✓ [+10,+33]	1.000 [−3,+7]	0.508 [−5,+15]	<.001✓ [+10,+30]	0.146 [−2,+22]	1.000 [−14,+23]

satisfy a target average in-degree of 2.0, and injecting one or more known faulty assumptions whose downstream effects propagate to a deterministic verifier. The verifier checks whether the trajectory output satisfies a set of compositional constraints derived from the gold graph. We generate 60 such graphs per condition. In `single_fault`, exactly one ancestor is faulty; in `multi_valid`, multiple ancestors can each independently resolve the violation but at different downstream edit costs.

I PROFILE-X CONTROLLED BENCHMARKS

Profile-X attaches structured user profiles to canonical tasks under our control and injects known assumption faults. Hyperparameters were tuned on the Profile-X validation split. Profile-X numbers are reported as supplementary evidence; the main paper emphasizes Profile-injected-X (§3) and TravelPlanner (Appendix J).

J TRAVELPLANNER RESULTS

Table 19: Real-TravelPlanner ($n=36$ per cell after merging seeds; success rate %).

	Direct	SR	Rfx	FR	ReAct	SChk	ToT	INTRO-SPECTER
Llama 8B	52.8	52.8	80.6	47.2	52.8	<u>74.2</u>	63.6	66.7
Llama 70B	25.0	22.2	25.0	25.0	41.7	34.5	36.1	33.3
Mistral 12B	25.0	27.8	41.7	19.4	<u>36.1</u>	22.6	0.0	27.8
Qwen 7B	41.7	41.7	66.7	38.9	<u>52.8</u>	34.4	36.1	54.3

INTRO-SPECTER records one numerical win on Llama 70B (33.3% vs. Reflexion 25.0%) and three NS losses (all $p \geq 0.180$). Short trajectories with few model-inferred assumption nodes reduce the value of structured attribution.

Calibration to the official TravelPlanner evaluator. We caveat the absolute numbers above. Our success criterion is a permissive subset of the official TravelPlanner evaluator: we apply only the dataset’s hard-constraint predicates (budget, allergies, transportation rules, room counts, and accommodation/cuisine includes) plus our injected profile-violation predicates. We do *not* apply the official evaluator’s commonsense-constraint pass (e.g., realistic transit times, consistent attraction-to-attraction scheduling, no overnight transit between cities) nor the environmental-constraint pass (validity of the returned restaurants, attractions, and accommodations against the dataset’s reference databases). The official evaluator’s *Final Pass Rate* requires all three layers (hard + commonsense + environmental) to pass simultaneously; under that stricter rubric, recent specialized planners (ATLAS Choi et al. (2025): 44.4%, Behavior Forest Yuan et al. (2026): up to 94.4%, TriFlow Chen et al. (2026): 91.1%) substantially outperform any general-purpose self-correction baseline including INTRO-SPECTER.

The 80.6% Reflexion rate on Llama 8B in our table should not be read as a state-of-the-art TravelPlanner result—it reflects only that, on the subset of constraints we score, Reflexion’s full-trajectory regeneration tends to satisfy the explicit hard constraints we retained. This appendix is included to complete the per-method comparison on user-conditioned benchmarks, not to claim a TravelPlanner accuracy result. The relevant comparison is the within-row Δ between methods evaluated on the identical predicate set; the absolute level is uncalibrated against the official benchmark.

K FAILURE-CASE EXAMPLES

Representative example trajectories per failure bucket (verifier miss, wrong edge, wrong prior, counterfactual failure, no valid repair) are included in supplementary materials; see the released code repository for the full case-study notebook.

L M -SWEEP DETAIL

To examine the non-obvious finding that $M=1$ outperforms $M=3$, we extend the sweep to $M \in \{1, 3, 5\}$ on the four ablation cells. Holding all other components at their default values, the mean success rate decreases at $M=3$ and remains below the $M=1$ baseline at $M=5$, but the per-cell pattern is non-monotone (Table 20).

Table 20: M -sweep on the four ablation cells. Cells are absolute success rate (%); right column is the mean delta vs $M=1$. The TruthfulQA cells use $n=36$ paired trials per cell at $M=5$ due to API-budget exhaustion during the sweep (other rows use the full $n=60$ or $n=180$).

M	TruthQA $\times$ Mist	TruthQA $\times$ Qwen	2Wiki $\times$ Mist	LongMem $\times$ Llama8B	Mean Δ
1 (default)	86.1	77.8	96.7	78.3	0.0
3	80.6	66.7	96.7	75.0	-5.0
5	72.2	66.7	96.7	78.3	-6.3

The mean trend is decreasing in M but the per-cell pattern is non-monotone, which is the diagnostic signal we discuss in §4.5. Per-cell behavior: TruthfulQA $\times$ Mistral shows a monotone decrease ($86.1 \rightarrow 80.6 \rightarrow 72.2$); TruthfulQA $\times$ Qwen drops at $M=3$ then plateaus; 2Wiki $\times$ Mistral is at the saturation ceiling and does not move; **LongMemEval $\times$ Llama 8B drops at $M=3$ then recovers to the $M=1$ value at $M=5$** . The recovery on LongMemEval is the diagnostic that rules out a simple i.i.d. Bernoulli interpretation: under that model, $\hat{r}_i$ would have variance $\theta_i(1 - \theta_i)/M$, monotone-decreasing in M , and accuracy should improve (or at minimum not recover non-monotonically) as M grows. The pattern is more consistent with the within-candidate trials being correlated rather than independent (shared prompt context, shared decoding trace), so larger M does not realize the variance reduction the i.i.d. model would predict and instead introduces small misranking events on cells with closely matched priors (TruthfulQA), while having negligible net effect on cells where the prior already strongly discriminates (2Wiki and LongMemEval). Variance-reduced or seed-diversified counterfactual sampling are the natural next steps; we use $M=1$ as the empirical operating point of the current framework.